

Working with Strings in C

Initialization, Functions, and Memory Behavior

Mohammad Maksood Akhter, PhD
Assistant Professor

Department of Computer Science & Engineering

October 6, 2025



Note

In the C language, a **string** is a sequence of characters terminated by a special character called the **null character** (`\0`). Strings are stored in arrays of type `char`.

- A string in C is represented as a `char` array.
- Every string in C is terminated with a **null character** (`\0`), ASCII value 0.
- Anything in double quotes (" ") is a **string constant**.

Example 1 – String as a Character Array

```
1      char s[10] = "INDIA";
```

Example 2 – String Constant with Pointer

```
1      char *s = "INDIA";
```

Array version is modifiable, pointer version points to a constant string literal.

Tip

The **name of a string (array name)** represents the *base address* of its first character. For example: `printf("%s", str);` prints from the base address automatically.

Example 3 – Printing Characters One by One

```
1  #include <stdio.h>
2  void main() {
3      char str[20] = {'a','b','c','d','e','\0'};
4      for(int i=0; str[i]!='\0'; i++)
5          printf("%c ", str[i]);
6  }
```

Output: a b c d e

Note

Each character is stored in consecutive memory locations. The loop prints characters until it reaches the null terminator (`\0`).

Example 4 – Printing a String Using %s

```
1      #include <stdio.h>
2      void main() {
3          char str[20] = "abcd";
4          printf("%s", str);
5      }
```

Output: abcd

Note

When printed using %s, printf() starts from the base address of str and continues until it encounters the null character (\0).

Example 5 – Memory View of a String

```
char S[6] = "ABCD";
```

Memory Representation

Index	0	1	2	3	4
Character	A	B	C	D	\0
ASCII	65	66	67	68	0

If S starts at address 100, then:

$$S[0] = 100, S[1] = 101, S[2] = 102, S[3] = 103, S[4] = 104$$

The string occupies 5 bytes (4 characters + 1 null terminator).

Tip

Accessing `S[i]` gives the character at that position, which corresponds to its ASCII value.
Example: `S[1]` has value 66 because `'B' = 66`.

Key Takeaways

- Strings in C are arrays of characters ending with a null character (`\0`).
- Array name acts as a pointer to the first character.
- Using `%s` prints characters until the null character.
- Characters are stored sequentially in contiguous memory.

Note

A string in C can be initialized in multiple ways. The method determines whether the string is stored as a **modifiable character array** or as a **string constant** in read-only memory.

Example

Examples:

```
1      char s[6] = "INDIA";           // Fixed-size array
2      char s[] = "INDIA";            // Size auto-counted
3      char S[6] = {'I','N','D','I','A','\0'}; // Element-wise
4      char S[] = {'I','N','D','I','A','\0'}; // Size inferred
5      char *p  = "HELLO";            // String constant pointer
```

Note

A string in C can be initialized in multiple ways. The method of initialization determines whether the string is stored as a **modifiable character array** or as a **string constant** in read-only memory.

Example

Examples of Initialization:

```
1      char s[6] = "INDIA";           // Fixed-size array
2      char s[] = "INDIA";           // Size inferred by compiler
3      char S[6] = {'I','N','D','I','A','\0'}; // Element-wise
4      char S[] = {'I','N','D','I','A','\0'}; // Size inferred
5      char *p = "HELLO";             // String constant pointer
```

Tip

Char Array vs String Constant:

```
1      char s[6] = "INDIA";    // modifiable array
2      char *p = "INDIA";      // read-only pointer
```

Aspect	Char Array	String Constant (Pointer)
Declaration	<code>char s[6] = "INDIA";</code>	<code>char *p = "INDIA";</code>
Storage	Stack memory (modifiable)	Read-only memory
Modifiability	Allowed	Not allowed (runtime error)
Example	<code>s[1]='A';</code>	<code>p[1]='A';</code> Not allowed
Base Address	Fixed	Pointer can be reassigned

Important

Key Insights:

- Character arrays are **modifiable**, but the base address cannot be reassigned.
- String constants (`char *p`) point to **read-only memory**.
- Modifying a string constant causes a segmentation fault.
- Array size is always one more than the number of visible characters (for `\0`).

Example

Character Array Example:

```
1      #include <stdio.h>
2      void main() {
3          char s[6] = "INDIA";
4          s[1] = 'O'; // Modify second character
5          printf("%s", s);
6      }
```

Output: IODIA

Example

String Constant Example:

```
1      #include <stdio.h>
2      void main() {
3          char *p = "INDIA";
4          p[1] = '0'; // Not allowed, segmentation fault
5          printf("%s", p);
6      }
```

Error: Attempt to modify read-only memory

Note

Declaration:

```
1 char s[] = "INDIA";
```

Memory layout (stack, modifiable):

Index	0	1	2	3	4
Character	I	N	D	I	A
ASCII	73	78	68	73	65

A hidden sixth byte stores the null terminator `\0`.

Tip

- Omitting size (e.g., `char s[] = "HELLO";`) lets compiler allocate space for all characters plus `\0`.
- Array name (`s`) represents the starting address.
- Pointer initialization (`char *p = "HELLO";`) stores only the address of the string literal.

Important

- Character arrays are **modifiable**, but their base addresses cannot be reassigned.
- String constants (`char *p`) point to **read-only memory**.
- Attempting to modify a string constant leads to a segmentation fault.
- Array size is always one more than the visible characters (to store `\0`).

Example 1 – Pointer and Array Assignments (Code)



```
1      #include <stdio.h>
2
3      void main() {
4          char str1[] = "INDIA";
5          char str2[20];
6          char *s = "INDIA";
7          char *q;
8
9          str2 = str1;    // Not allowed
10         q = s;          // Pointer assignment allowed
11
12         printf("String s: %s\n", s);
13         printf("String q: %s\n", q);
14     }
```

Example 2 – Comparing Strings with == (Code)



```
1  #include <stdio.h>
2
3  int main() {
4      char str1[10] = "hello";
5      char str2[10] = "hello";
6
7      if (str1 == str2) {
8          printf("Strings are equal\n");
9      } else {
10         printf("Strings are different\n");
11     }
12
13     return 0;
14 }
```

Output: Strings are different

Note

Explanation: The `==` operator compares the **base addresses**, not content. To compare contents, use `strcmp(str1, str2) == 0`.

Example 3 – Comparing First Characters (Code)



```
1      #include <stdio.h>
2
3      int main() {
4          if (*"hello" == *"hail") {
5              printf("They are same\n");
6          } else {
7              printf("They are different\n");
8          }
9          return 0;
10     }
```

Output: They are same

Note

Explanation: `*"hello"` and `*"hail"` dereference the first character only (`'h'`). Since ASCII values match, the comparison is true. Other characters are ignored.

Example 4 – Pointer Arithmetic (Code)



```
1  #include <stdio.h>
2  int main() {
3      char str[20] = "abcdef";
4      printf("%s\n", str);
5      printf("%s\n", str + 3);
6      str[3] = 97;
7      printf("%s\n", str);
8      str[3] = '\\0';
9      printf("%s\n", str);
10     str[3] = 0;
11     printf("%s\n", str);
12     str[3] = 'd';
13     printf("%s\n", str);
14     return 0;
15 }
```

Example 4 – Pointer Arithmetic (Outputs)



```
1  #include <stdio.h>
2  int main() {
3      char str[20] = "abcdef";
4      printf("%s\n", str);           // "abcdef"
5      printf("%s\n", str + 3);      // from index 3: "def"
6      str[3] = 97;                  // ASCII 'a'
7      printf("%s\n", str);          // "abcaef"
8      str[3] = '\0';                // null terminator
9      printf("%s\n", str);          // "abc"
10     str[3] = 0;                   // another way
11     printf("%s\n", str);          // "abc"
12     str[3] = 'd';                 // restore
13     printf("%s\n", str);          // "abcd"
14     return 0;
15 }
```

Note

- `str + 3` points to `str[3]`.
- Printing from `str + 3` starts output from that character onward.
- Assigning `'\0'` inserts a null terminator and ends the string.
- Replacing characters directly is allowed because `str` is modifiable stack memory.

Important

- Arrays cannot be assigned directly (`str2 = str1`; not allowed).
- `==` compares addresses, not contents.
- Dereferencing string literals (`*"abc"`) compares only the first character.
- Pointer arithmetic (`str + n`) accesses substrings from the `n`th index.
- Null terminator (`\0`) defines the logical end of a string.

Overview of String library (`#include<string.h>`) and its Functions



The C standard library provides a collection of useful functions for manipulating strings. These functions are declared in the header file `<string.h>`. They handle common operations like finding length, copying, concatenation, and comparison.

Function	Purpose
<code>strlen()</code>	Finds the length of a string
<code>strcpy()</code>	Copies one string into another
<code>strcat()</code>	Concatenates (joins) two strings
<code>strcmp()</code>	Compares two strings
<code>strncpy()</code>	Copies <i>n</i> characters from source to destination
<code>strncat()</code>	Concatenates <i>n</i> characters from source to destination
<code>strchr()</code>	Finds first occurrence of a character
<code>strstr()</code>	Finds first occurrence of a substring

- All functions are declared in `<string.h>`.
- Strings must be null-terminated (`\0`).
- Functions like `strcpy` and `strcat` require sufficient memory in the destination array.
- Use `strncpy`, `strncat` to prevent buffer overflow.
- Comparison functions (`strcmp`) return 0 if strings are equal, negative if first string is smaller, positive if larger.

1. strlen()

```
1 char name[] = "SOA University";  
2 int len = strlen(name);  
3 printf("%s -> %d", name, len);
```

Output & Tip:

1. strlen()

Output: SOA University – > 14

Tip: Counts characters until null terminator `\0`, not included.

2. strcpy()

```
1 char src[] = "C Programming";  
2 char dest[50];  
3 strcpy(dest, src);  
4 printf("%s -> %s", src, dest);
```

2. strcpy()

Output: C Programming – > C Programming

Tip: Destination array must be large enough to avoid overflow.

3. strcat()

```
1 char first[50] = "Hello ";  
2 char second[] = "World!";  
3 strcat(first, second);  
4 printf("%s", first);
```

3. strcat()

Output: Hello World!

Tip: Appends src to dest; null terminator added automatically.

4. strcmp()

```
1 char a[] = "apple";
2 char b[] = "banana";
3 printf("%d", strcmp(a,b));
```

5. sizeof vs strlen

```
1 char str1[] = "ITER";
2 char str2[20] = "ITER";
3 printf("%lu%lu", sizeof(str1), strlen(str1));
4 printf("%lu%lu", sizeof(str2), strlen(str2));
```

6. strchr() / strstr()

```
1 char text[] = "C programming";
2 char *p1 = strchr(text, 'p');
3 char *p2 = strstr(text, "gram");
4 printf("%s%s", p1, p2);
```

Output & Tip:

4. strcmp()

Output: -1 (a<b)

Tip: 0 if equal, < 0 if first < second, > 0 if first > second.

5. sizeof vs strlen

Output: sizeof(str1)=5, strlen(str1)=4
sizeof(str2)=20, strlen(str2)=4

Tip: sizeof = allocated memory, strlen = actual characters.

6. strchr() / strstr()

Output: First occurrence of 'p':
programming

Substring found: gram

Tip: Returns pointer to first occurrence or NULL if not found.

Thank You

Questions?